

The background features a decorative geometric design. It includes several white diamonds of various sizes, some of which are outlined in black. These diamonds are arranged in a way that they appear to be floating or connected by thin black lines. Additionally, there are small, dark, four-pointed star-like shapes scattered throughout the design, particularly near the corners and the central text area. The overall aesthetic is clean and modern, with a focus on geometric shapes and negative space.

MANUAL DE USUARIO

Índice

Instalación del proyecto.....	4
Conexión de red local.....	4
Configuración del servidor.....	4
Configuración de dirección IP.....	4
Configuración de variables.....	5
Configuración de cliente.....	5
Configuración de dirección IP.....	5
Configuración de variables.....	6
Conexión Cliente-Servidor.....	6
Calibración.....	7
Versiones de librerías.....	7
Parámetros.....	7
Sobre la rutina.....	8
Ejecución.....	8
Segmentación.....	9
Consideraciones iniciales.....	9
Entorno.....	9
Tópicos y parámetros.....	10
Lógica de selección del algoritmo.....	11
Ejecución.....	12
Transformación.....	12
Tópicos y parámetros.....	12
Ejecución.....	13
Movimiento y telemetría.....	13
Ejecución.....	13
Máquina de estados.....	13
Variables globales.....	13
Ejecución.....	14
Estado HOME.....	14

Estado ESPERAR_PUNTO	15
Estado TRANSFORMAR_PUNTO	15
Estado MOVER_A_CENTROIDE	15
Estado BAJAR_EN_Z.....	16
Estado SUBIR_EN_Z.....	17
Estado ALIMENTAR	17
Simulación.....	18
Modelos 3D	18
Lanzamiento	19
Segmentación	20
Movimiento	21
Máquina de estados	21

Instalación del proyecto

El proyecto se encuentra almacenado en el siguiente repositorio público de GitHub.

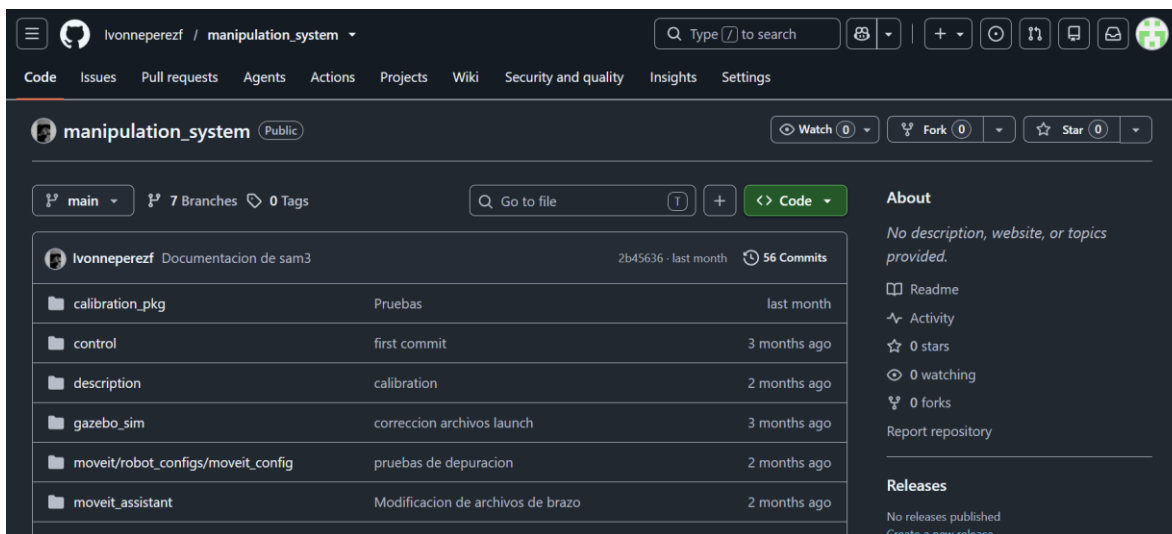


Figura 1. Repositorio remoto de manipulation_system

Enlace al repositorio: https://github.com/Ivonneperezf/manipulation_system

Para descargar el contenido se ejecuta el siguiente comando (debe realizarse dentro del workspace donde se instaló previamente el repositorio de kinova [driver kinova](#) y el repositorio de realsense [realsense driver](#)):

```
git clone https://github.com/Ivonneperezf/manipulation_system.git  
manipulation_system
```

Posteriormente, compilar el proyecto de manera habitual.

Conexión de red local

El propósito de la conexión de un cliente a un mismo roscore es mejorar el procesamiento y el uso de la tecnología de SAM3. Para llevar a cabo esta tarea es necesario realizar algunas configuraciones previas.

Configuración del servidor

Configuración de dirección IP

Para configurar la conexión de servidor debemos de entrar a los ajustes de red. Una vez dentro localizamos la pestaña IPv4, donde nos aseguraremos de que el equipo esté conectado a la red *carrito*. Posteriormente en el campo *IPv4 Method* seleccionamos la opción *manual* y en el campo *Addresses* colocamos la dirección:

192.168.0.200

Después en el campo *Netmask* colocamos la máscara de red:

255.255.255.0

The screenshot shows the Mikrotik WinBox configuration window for the 'carrito' interface. The 'IPv4' tab is selected. Under 'Método IPv4', the 'Manual' option is chosen. The 'Direcciones' table has two rows: the first row contains '192.168.0.200' for the address and '255.255.255.0' for the netmask. The 'DNS' section has a toggle for 'Automático' which is turned on. The 'Rutas' section also has a toggle for 'Automático' which is turned on.

Dirección	Máscara de red	Puerta de enlace
192.168.0.200	255.255.255.0	

Figura 2. Configuración de red del servidor

Configuración de variables

Para evitar realizar configuraciones por terminal podemos incluirlas en el archivo `~/bashrc` (si se usa ROS de manera local comentar las líneas en el archivo `bashrc`).

```
export ROS_MASTER_URI="http://192.168.0.200:11311/"
export ROS_IP=192.168.0.200
```

Configuración de cliente

Configuración de dirección IP

La configuración es similar a la configuración del servidor, con la diferencia de que esta vez usamos la siguiente dirección IP:

192.168.0.100

Usamos la misma máscara de red.

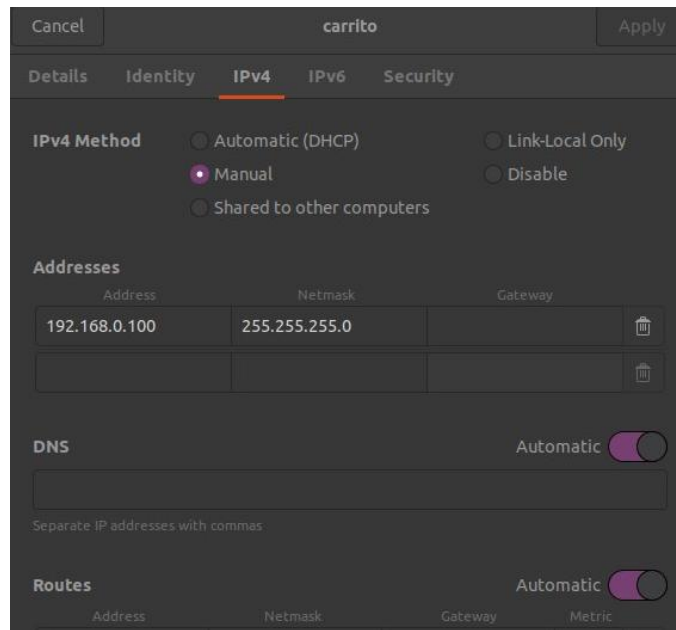


Figura 3. Configuración de red del cliente

Configuración de variables

La configuración de las variables esta almacenada en un archivo sin extensión disponible en la carpeta del usuario llamado *arm*. Este archivo cuenta con la misma configuración del servidor. Para usarlo ejecutamos el siguiente comando por terminal:

```
source arm
```

```
(robot_env) lleal@allseries:~$ cat arm
source /opt/ros/noetic/setup.bash
export ROS_MASTER_URI="http://192.168.0.200:11311/"
export ROS_IP=192.168.0.100
```

Figura 4. Configuración de variables del cliente

Conexión Cliente-Servidor

Para realizar la conexión entre ambos equipos se debe tener en ejecución un *ROS master* en el equipo que ejerce el rol de servidor. Para verificar la conexión del cliente, ambos equipos deben tener el mismo workspace y estar sourceados.

Podemos notar que la conexión ha sido exitosa, podemos ejecutar cualquier comando que visualice tópicos o nodos.

Calibración

Para realizar la calibración es necesario descargar e imprimir uno de los arucos disponibles en el rango de 0 a 250 de 4x4, disponibles en el enlace <https://chev.me/arucogen/>.

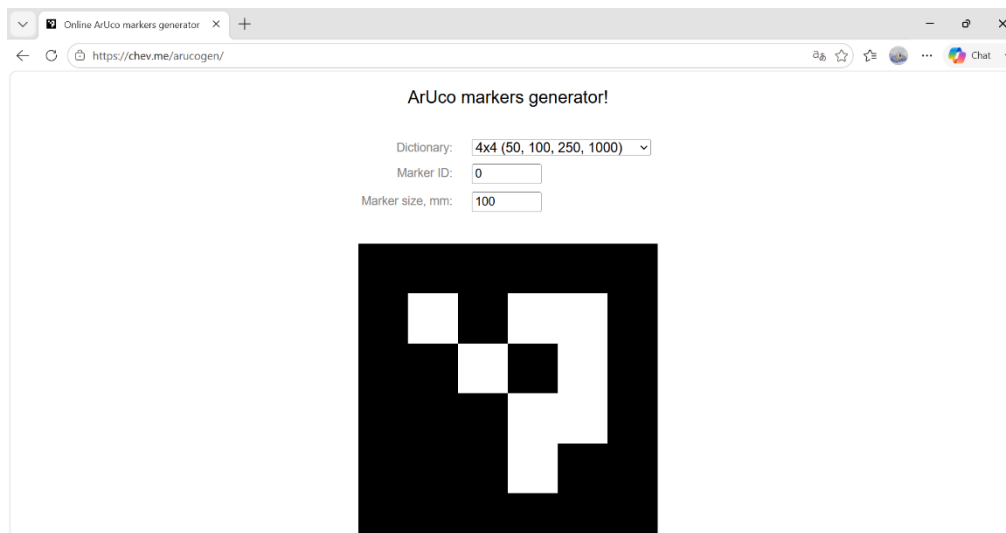


Figura 5. Generador de arucos.

Nota. El tamaño una vez impreso puede variar, por lo que es necesario realizar la medición correcta una vez impreso.

Versiones de librerías

Debemos asegurarnos de que las librerías para realizar la ejecución de la calibración sean las siguientes:

- numpy 1.17.4
- opencv 4.13.0.92

Parámetros

Dentro del archivo *handeye_calibration.py* encontramos algunos parámetros que se deben ajustar de acuerdo con la calibración, los que podemos ver a continuación:

Nombre del parámetro	Descripción	Valor
base_frame	Define la base del brazo.	m1n6s300_link_base
ee_frame	Define el eslabón al que se encuentra fija la cámara.	m1n6s300_link_5
n_captures	Define el número de poses que se realizarán durante la rutina de calibración.	20

size	Define el tamaño del aruco (expresado en metros).	0.108
------	---	-------

Sobre la rutina

La calibración es almacenada en el paquete de calibración en la carpeta *config* en dos formatos, como matriz homogénea, formato npz y como vectores de traslación y rotación en formato YAML.

Algunas de las consideraciones que se deben de tomar en cuenta durante la calibración es que el aruco se encuentre fijo a no mas de 1.5 metros de la base del robot y permitiendo libertad de movimiento en el eje z. Además, las poses deben de ser diferentes entre sí, variando tanto la traslación como orientación de todas las articulaciones del brazo.

Ejecución

Antes de iniciar la ejecución del nodo debemos tener conectado tanto la cámara como el brazo a la PC, y corriendo sus nodos con los siguientes comandos:

```
# Correr el driver del brazo
roslaunch kinova_bringup kinova_robot.launch
kinova_robotType:=m1n6s300

# Correr el driver de la cámara
roslaunch realsense2_camera rs_camera.launch
```

Para ejecutar la rutina de calibración se debe escribir el siguiente comando en terminal:

```
roslaunch calibration_pkg handeye_calibration.py
```

Esto desplegará las indicaciones y el conteo de capturas tomadas al momento.

```
user@user:~/Documentos/ROS/projects/kinova_robot_ws$
un calibration_pkg ha
ndeye_calibration.py
[INFO] [1784136520.107860]: Esperando camera_info...
[INFO] [1784136520.136313]: Intrínsecos cargados desde camera_info

[0/20] Mueve el robot a una nueva pose y presiona ENTER para capturar...[
```

Figura 6. Ejecución de rutina de calibración

Para capturar la pose nos debemos asegurar que no se muestre ninguna advertencia en terminal, de ser así ajustar la pose hasta que la vista del aruco sea correcta como se muestra en la figura 7.

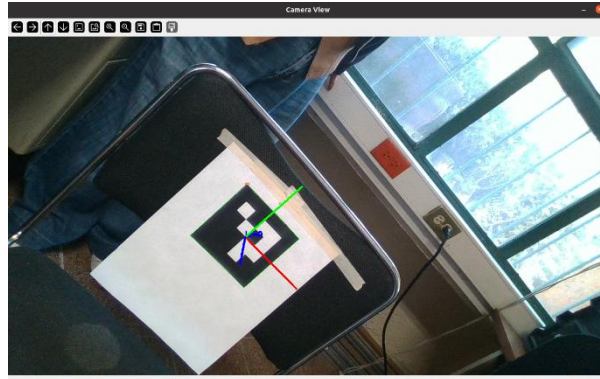


Figura 7. Vista del aruco en posición valida

Posteriormente presionar enter cuando la pose sea completamente valida.

Segmentación

Consideraciones iniciales

Para poder ejecutar el nodo de segmentación debemos de tener previamente configurado la red y corriendo un solo maestro en el servidor (tomar en cuenta que en el equipo localizado en el laboratorio no se corre el maestro).

Debemos cerciorarnos de que el modelo *sam3.pt* este en el paquete *statemachine* en la carpeta *weights*. Actualmente el modelo ya se encuentra en la computadora del laboratorio, en caso contrario este debe descargarse de <https://huggingface.co/facebook/sam3>.

Entorno

Para poder correr las librerías necesarias se debe activar el entorno *robot_env*.

```
conda activate robot_env
```

Este entorno cuenta con las versiones de librerías que se muestran en la figura 8.

```
(robot_env) lleal@allseries:~$ pip freeze | grep -iE "numpy|opencv|ultralytics|ros-numpy|rospkg"
clip @ git+https://github.com/ultralytics/CLIP.git@81ff68ed7ffcac3b40484c914f104f816757308d
numpy==1.23.5
opencv-python==4.13.0.92
ros_numpy==0.0.5
rospkg==1.6.1
ultralytics==8.4.23
ultralytics-thop==2.0.19
```

Figura 8. Versiones de entorno robot_env

Tópicos y parámetros

Este nodo publica sobre dos tópicos:

Nombre del tópico	Descripción
object_centroid	Se publica el punto centroide en un mensaje de tipo PointStamped.
object_pointcloud	Publica la nube de puntos filtrada de la máscara de segmentación en un mensaje de tipo PointCloud2.

Dentro del código *segmentation_sam3_single_piece.py* podemos encontrar dos clases. La clase *MaskFilter* determina el filtro para seleccionar una sola máscara, de manera que seleccione un solo cubo. En esta clase hay cinco parámetros ajustables:

Nombre del parámetro	Descripción	Valor
min_area	Define el área mínima que debe contener una máscara en píxeles.	100
max_area_ratio	Representa la fracción máxima del área de la imagen permitida.	0.04 (4%)
min_compactness	Determina que tan irregular es una máscara con relación a su bbox. Representa la relación mínima área máscara/área bbox.	0.10
contaiment_thresh	Proporción mínima de i dentro de j para eliminar j (subir este valor evita eliminar cubitos adyacentes)	0.92
size_ratio_min	j debe ser al menos esta vez mas grande que i para ser candidato a “contenedor”.	3.0

Dentro de este mismo código en la clase *KinovaVisionSAM3*, localizamos dos parámetros que definen la prioridad de selección de los cubitos.

Nombre del parámetro	Descripción	Valor
CENTER_FRACTION	Fracción del grupo usada como candidatos centrales.	0.3 (30%)
FEW_THRESHOLD	Si hay $\leq$ FEW_THRESHOLD cubitos, ignorar los cubitos centrales y usar solo aquel cubito con menor z.	4

Tomemos en cuenta que una vez que las máscaras de SAM 3 se encuentran limpias y filtradas, se aplica una lógica adaptativa para seleccionar cuál de todos los cubitos segmentados debe ser el objetivo del brazo robótico, cubriendo de esta forma los casos extremos de finalización de tarea:

Lógica de selección del algoritmo

1. *Modo Normal (cuando hay más de 4 cubitos)*: Se calculan los centroides de cada máscara y se determina el centro de masa de todo el conjunto. Se toma el 30% de los elementos más cercanos a dicho centro para evitar los límites físicos del plato. De este subgrupo central, se selecciona el cubito que presente la menor coordenada de profundidad (menor Z) para aproximar la pinza con seguridad. Lo podremos visualizar con los contornos del cubito en color Verde.
2. *Modo Final (cuando quedan 4 o menos cubitos)*: Se desactiva la restricción del grupo central para evitar fallas en la detección cuando queden cubitos aislados o dispersos. El algoritmo pasa a un modo directo que selecciona el cubito accesible con menor valor en Z. Lo podemos visualizar con los contornos del cubito en color Amarillo.

Para definir los overrides del modelo definimos:

Nombre del parámetro	Descripción	Valor
conf	Umbral mínimo de confianza (Confidence Threshold) para filtrar las detecciones/segmentaciones válidas.	0.2
task	Define el tipo de tarea que realizará el modelo (en este caso, segmentación de objetos).	"segment"
mode	Especifica la modalidad de ejecución del predictor (inferencia/predicción sobre los datos de entrada).	"predict"
model	Ruta relativa o absoluta del archivo de pesos entrenados (weights) del modelo a cargar.	os.path.join(package_path, "weights", "sam3.pt")
iou	Umbral de intersección sobre unión (Intersection over Union) utilizado para la supresión de no máximos (NMS) y evitar detecciones duplicadas.	0.3
half	Habilita el uso de precisión media (FP16 / Float16) para acelerar la inferencia en GPU y reducir el uso de memoria VRAM.	True

Dependiendo del modo de ejecución se cargan los parámetros correspondientes a los tópicos que se encuentran disponibles en la carpeta *config* del paquete *statemachine* con extensión *yaml* en los archivos *robot_params* y *simulation_params*.

Ejecución

Para ejecutar el nodo de segmentación es necesario tener previamente activo el nodo de la cámara con los argumentos de alineación y nube de puntos activados.

```
roslaunch realsense2_camera rs_camera.launch align_depth:=true  
filters:=pointcloud pointcloud_texture_stream:=RS2_STREAM_COLOR
```

Para lanzar el nodo ejecutar el comando:

```
roslaunch statemachine segmentation_SAM3.launch  
use_simulation_params:=false
```

Nota. En caso de error de la cámara, simplemente volvemos a correr la cámara en el dispositivo donde este conectada.

Nota. El proceso de carga del modelo puede tardar unos minutos.

Transformación

Este nodo resuelve la cadena cinemática del punto centroide obtenido por el nodo de segmentación a la base del brazo, usando la calibración previamente hecha. Siguiendo los puntos de referencia de la cámara al quinto eslabón, y del quinto eslabón a la base.

Tópicos y parámetros

Los parámetros usados hacen referencia a los frames del brazo en el eslabón donde se encuentra montada la cámara y la base del robot, además de obtener la matriz de calibración proveniente del paquete de calibración, y publica en el tópico *object_centroid_robot* el resultado de la transformación.

Nombre del parámetro	Descripción	Valor
base_frame	Punto de referencia cero del brazo.	m1n6s300_link_base
ee_frame	Eslabón donde se encuentra la montura de la cámara (debe corresponder al eslabón descrito en la calibración)	m1n6s300_link_5

Ejecución

Este nodo debe lanzarse con el siguiente comando:

```
roslaunch statemachine transform_node.launch
```

Movimiento y telemetría

Estos nodos se encuentran cargados en el mismo archivo de lanzamiento.

El nodo de movimiento se mantiene escuchando por medio de los tópicos *cartesian_goal* y *joint_goal*, que refieren al movimiento cartesiano y articular del brazo respectivamente. Posteriormente publica un mensaje de confirmación de movimiento en el tópico *motion_done* una vez que el movimiento fue resuelto.

Para el nodo de telemetría se mantiene publicando mensajes de tipo Float64MultiArray en los tópicos *kinova_current_joints* y *kinova_current_cartesian* con una frecuencia de 10 Hz, los cuales obtienen información de los tópicos */m1n6s300_driver/out/joint_command* y */m1n6s300_driver/out/cartesian_command* respectivamente y los descompone con el propósito de simplificar la obtención del estado actual del brazo con el objetivo de simplificar su uso en la máquina de estados.

Ejecución

Para lanzar estos nodos, ejecutamos el comando:

```
roslaunch statemachine move_node_driver.launch
```

Máquina de estados

Este nodo se encarga de dar secuencia a la rutina completa y consta de siete estados que definen un comportamiento basado en los demás nodos.

Variables globales

Las siguientes variables describen los valores offset a considerar durante la rutina.

Nombre de la variable	Descripción
TENEDOR	Tamaño del tenedor en centímetros.
OFFSET_Z	Pequeña distancia considerada previa y posteriormente a el objetivo.
OFFSET	Es la suma de offset_z y tenedor.

Ejecución

Para comenzar la ejecución de la rutina ya deben de estar lanzados los nodos previos. El comando para lanzar la máquina de estados es:

```
roslaunch statemachine statemachine_nodes.launch
```

Estado HOME

El estado home define una posición segura de lectura. En su mayoría de las veces resuelve correctamente poses calculadas a partir de esta pose. Consta de dos transiciones posibles, *Done* (movimiento completado correctamente) y *Failed* (Movimiento fallido). Publica sobre dos posibles tópicos *cartesian_goal* y *joint_goal* (el nodo *move_node_kinova* se suscribe a estos dos últimos tópicos) que definen el movimiento hacia la pose segura.

La posición esta definida en la variable *home_joint_goal.data* que define la posición en radianes de cada articulación del brazo. Esta se define de manera manual obteniendo los valores desde terminal. Esta información es publicada en el tópico *joint_goal* para posteriormente, esperar el mensaje de confirmación proveniente del tópico *motion_done*.

Para el caso de éxito del movimiento, se realiza la transición al siguiente estado *ESPERAR_PUNTO*, en caso contrario, vuelve a intentar alcanzar la pose Home.



Figura 9. Robot en pose HOME

Estado ESPERAR_PUNTO

Este estado se encarga de recibir el punto centroide del cubo detectado proveniente del nodo de segmentación, y publicado en el tópico *object_centroid* para después transmitirlo al siguiente estado por medio de los datos de usuario (userdata).

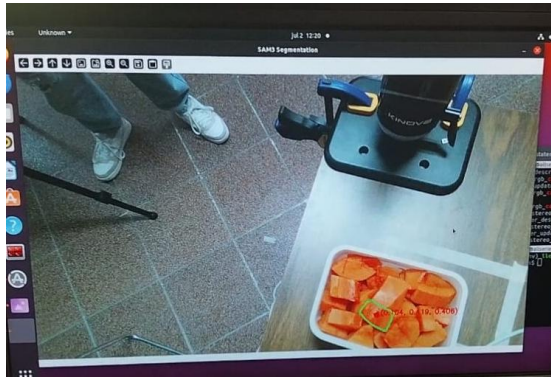


Figura 10. Visualización de punto centroide del nodo de segmentación

Estado TRANSFORMAR_PUNTO

Recibe el punto del centroide del estado anterior y lo publica en el tópico *object_centroid_sm* para, posteriormente, esperar el punto transformado a coordenadas del robot por medio del tópico *object_centroid_robot* publicado por el nodo de transformación. Esta información es transmitida por medio de los datos de usuario al siguiente estado.

Estado MOVER_A_CENTROIDE

Este estado utiliza el nodo de movimiento previamente usado. Publica el punto transformado en el tópico *cartesian_goal* construyendo un punto de tipo *PointStamped* sumando el offset a el valor transformado de Z, debido a la definición del sistema coordenado del brazo.

Nota. El estado cuenta con el uso del método *sleep* de 10 segundos a manera de seguridad, ya que en ocasiones la cámara realiza lecturas falsas que podrían afectar el punto objetivo. Se aconseja agregar un mecanismo de seguridad.

Finalmente, una vez realizado el movimiento se recibe un mensaje de confirmación proveniente del tópico *motion_done*.



Figura 11. Vista estado Mover a centroide ejecutado

Estado BAJAR_EN_Z

Este nodo se encarga de, una vez llegado al centroide considerando el offset, bajar el valor del OFFSET_Z hasta picar la fruta. Se estructura de manera similar a los demás estados de movimiento con la diferencia de que al estructurar el mensaje a enviar utiliza el nodo de telemetría para obtener la posición actual del brazo y restando el valor OFFSET_Z al valor de Z.



Figura 12. Ejecución del estado BAJAR_EN_Z

Estado SUBIR_EN_Z

Trabaja igual que el estado BAJAR_EN_Z, con la diferencia de que esta vez el valor OFFSET_Z es sumado al valor actual en Z. El propósito de este estado es asegurar una posición donde, al realizar movimientos posteriores, el tenedor no golpee el plato.



Figura 13. Ejecución del estado SUBIR_EN_Z

Estado ALIMENTAR

Este estado posiciona el brazo en una pose definida dentro del código, sin embargo, el propósito de este estado es continuar con estados correspondientes a la rutina de alimentación, por lo que este estado cuenta con un comportamiento con fines de demostración.



Figura 14. Ejecución del estado ALIMENTAR

Nota. Una sugerencia es agregar estados que gestionen las transiciones de error, ya que actualmente al obtener algún error de un estado automáticamente se reintenta la pose HOME.

Simulación

Para acceder a las carpetas de simulación es necesario cambiar de rama a la rama *simulation* con el comando:

```
git checkout simulation
```

Modelos 3D

Para poder visualizar los mundos creados, se deben descargar los modelos del siguiente repositorio https://github.com/Ivonneperezf/modelos_de_gazebo.

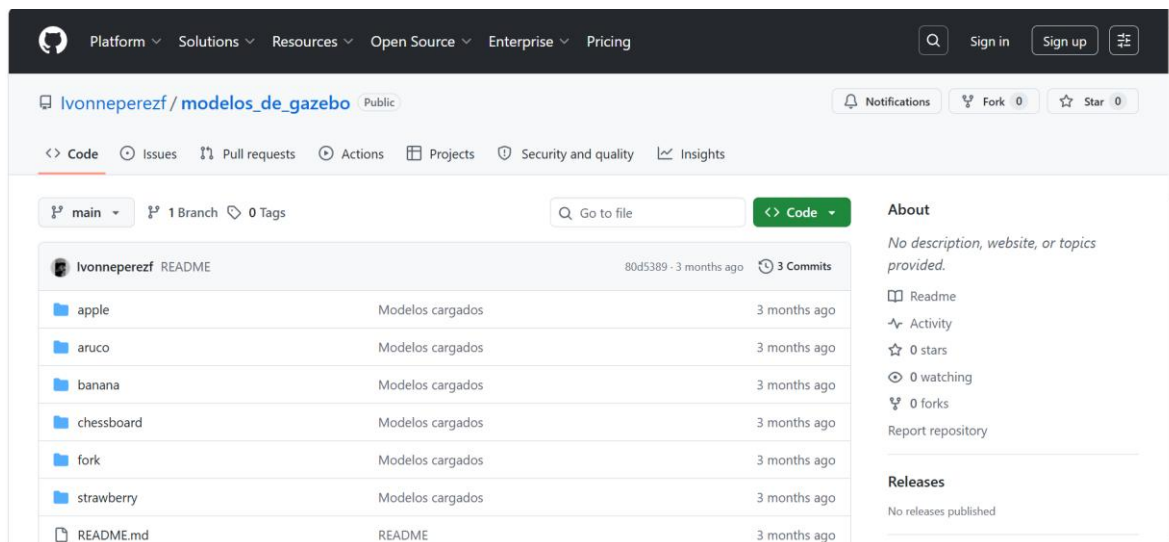


Figura 15. Repositorio de modelos de simulación

Se deben seguir los siguientes pasos:

1. Clonar el repositorio en una carpeta temporal de /tmp.

```
git clone https://github.com/Ivonneperezf/modelos_de_gazebo.git  
/tmp/modelos_gazebo
```

2. Copiar los modelos a ~/.gazebo/models

```
cp -r /tmp/modelos_gazebo/* ~/.gazebo/models/
```

3. Eliminar la carpeta temporal

```
rm -rf /tmp/modelos_gazebo
```

Lanzamiento

Para poder realizar la simulación es necesario estrictamente lanzar el nodo de Gazebo con el comando:

```
roslaunch gazebo_sim gazebo_kinova_sim.launch
```

Este archivo de lanzamiento permite utilizar diferentes mundos ubicados en la carpeta *worlds* del paquete *gazebo_sim* por medio del argumento *world*, este valor debe escribirse el nombre del mundo sin extensión.

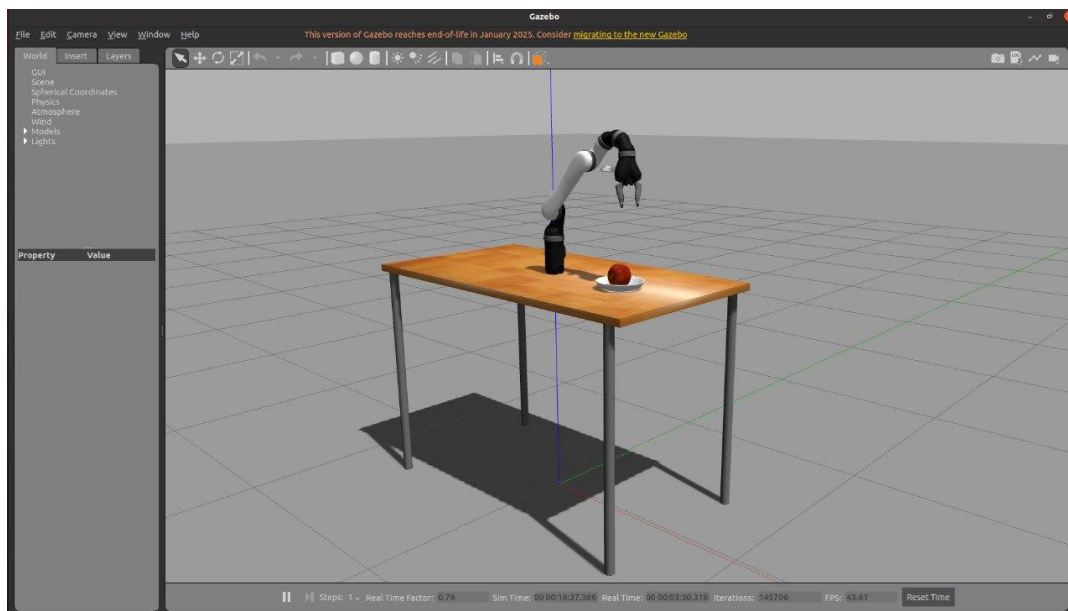


Figura 16. Simulación en Gazebo

Para lanzar Rviz se ejecuta el siguiente comando:

```
roslaunch gazebo_sim rviz_kinova_sim.launch
```

Esto cargará la pantalla de visualización de Rviz con los tópicos de la cámara y nube de puntos visualizados.

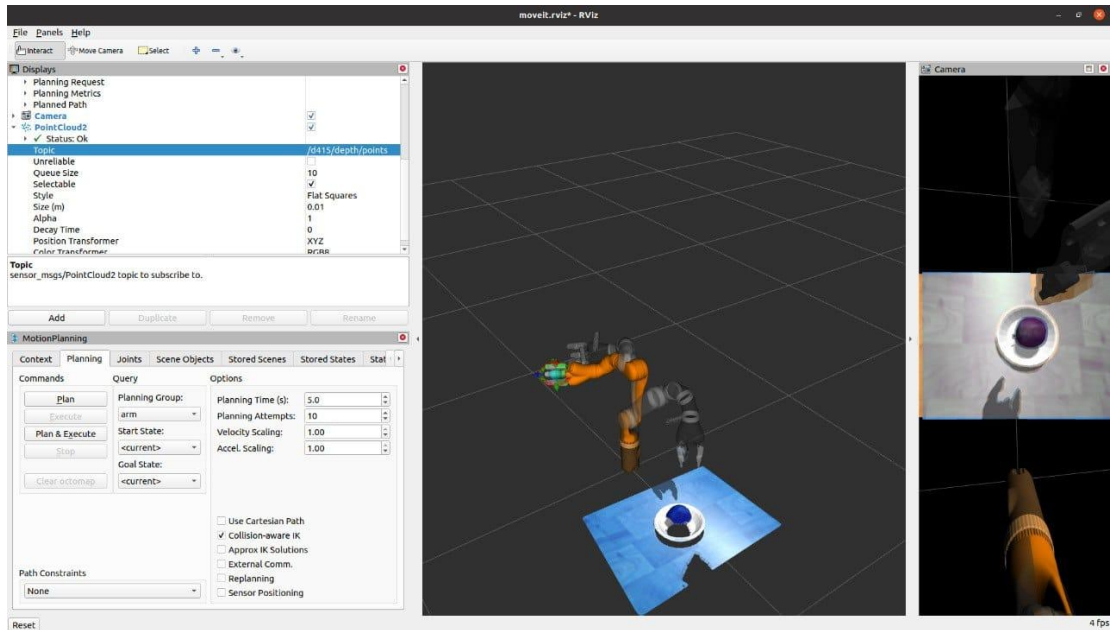


Figura 17. Pantalla de Rviz con simulación.

Segmentación

En este caso solo contamos con un nodo de transformación y uno de segmentación. El nodo de segmentación trata de una versión beta hecha con SAM2 en su versión base y YOLOv8 en su versión small, en este caso solo identifica las siguientes clases:

Clase	Nombre de la clase
0	Person
41	Cup
42	Fork
43	Knife
44	Spoon
45	Bowl
46	Banana
47	Apple
48	Sandwich
49	Orange
50	Broccoli
51	Carrot
52	Hot dog
53	Pizza
54	Donut

55	Cake
----	------

A partir de ella envía la máscara de la segmentación detectada a partir de la clase dada por el modelo de YOLOv8.

El comando para lanzar el nodo de segmentación es:

```
roslaunch statemachine segmentation.launch
```

Movimiento

El nodo de movimiento se encarga de resolver la cinemática inversa del brazo en simulación, usando la API *MoveIt!*. Para lanzar el nodo ejecutar:

```
roslaunch statemachine move_node.launch
```

Máquina de estados

Este nodo cuenta únicamente con tres estados, HOME, ESPERAR_PUNTO, MOVER_A_PUNTO, ya que la simulación solo muestra una versión inicial de la rutina.